Commutative m-ary Quasigroup Actions in a 2-adic Regime: A Standalone 2D CQA KEX Candidate, Perturbation Tests, and Open Problems

Danilo Gligoroski

Norwegian University of Science and Technology (NTNU)

danilog@ntnu.no

12 March 2026

Abstract

This note updates the current CQA project around a standalone two-dimensional key-exchange scaffold. The main construction starts from a scalar 2-adic commutative-quasigroup-action family over $R_w = \mathbb{Z}/2^w\mathbb{Z}$, then lifts it to a 2D swap-family operator

$$M_K = \lambda_0(K)I + \lambda_1(K)S, \qquad S = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

where the two scalar coefficients are extracted by reduced macro-step maps

$$\lambda_0(K) = T_{K,\tau_0}(xseed_0), \qquad \lambda_1(K) = T_{K,\tau_1}(xseed_1).$$

This version reflects the latest experiments on the current standalone script and its batch harnesses. The main empirical messages are: (i) the postponed rounds≥ 2 public-key bug is fixed; (ii) forcing the public base-vector coordinates g_1, g_2 to be odd blocks the obvious one-public-vector recovery condition based on $\Delta(G) = g_1^2 - g_2^2$; (iii) the severe collapse observed at tiny width $w = 16$ is a toy-width phenomenon; and (iv) randomized coefficient-extraction seeds $(xseed_0, xseed_1)$ do change the collapse landscape at $w = 16$, but they do not destroy the clean high-width odd-G regime, where no collapse was observed in long runs for widths $112 \leq w \leq 256$ and in particular for $w = 128$ up to 10^5 trials.

Keywords: commutative actions; hidden homogeneous spaces; m-ary quasigroups; 2-adic structure; key exchange; perturbation sweeps.

1 Introduction

Commutative actions generalize Diffie–Hellman-style key exchange beyond groups: parties apply secret actions to a common public base object, and commutativity guarantees that the two action orders lead to the same shared value. This is the Hidden-Homogeneous-Space (HHS) viewpoint introduced by Couveignes and later exemplified by CSIDH-style class-group actions on supersingular elliptic curves.[Cou06, CLM$^+$18, DF17]

The present project explores a different commutative-action candidate: an m-ary quasigroup/Latin-inspired action over the dyadic ring $R_w = \mathbb{Z}/2^w\mathbb{Z}$. The basic idea is to start from an m-ary local law and use a cyclic wiring to turn secret tuples into unary self-maps of the public state space. The current implementation status is a standalone research PoC, available as a public GitHub repository.[Gli26]

This version differs from the previous note in two important ways.

1. The current mainline design is now the explicit standalone 2D KEX scaffold implemented in `cqa_2d_kex.py`; the paper therefore describes the exact design that is currently tested, rather than an earlier schedule-centric exploratory framework.
2. The latest experiments add a *Campaign 3 perturbation sweep*: instead of fixing coefficient extraction at the anchor value $x = 1$, the design can use two public coefficient-extraction seeds $xseed_0, xseed_1 \in R_w$, allowing us to test whether the old anchor accidentally amplified or suppressed collapse.

The main empirical lesson from the latest experiments is now sharper than before. The 2D design behaves very differently in two regimes:

- at tiny width $w = 16$, collapse is common and the choice $x = 1$ materially biases the collapse-mode distribution;
- at moderate and large widths in the current odd-G regime, the design has remained clean in all current long runs, both in the anchored version and in the randomized-`xseed` perturbation variant.

2 Preliminaries

2.1 2-adic rings and units

Let

$$R_w := \mathbb{Z}/2^w\mathbb{Z}.$$

We write $v_2(x)$ for the 2-adic valuation of a representative of $x \in R_w$, and we denote the odd residues (the units) by

$$R_w^\times = \{x \in R_w : x \equiv 1 \pmod 2\}.$$

The project is explicitly 2-adic: parity, valuation, and controlled divisibility by powers of 2 are structural aspects of the action rather than implementation details.

2.2 Polynomial permutations on odd residues

A recurring source of intuition comes from polynomial permutations on odd residues modulo powers of two. Markovski–Šunić–Gligoroski give coefficient constraints under which a polynomial induces a permutation of the odd residues modulo 2^n.[MŠG10]

In the present work, these results are used only as design intuition rather than as a theorem about the full action. The conservative scalar family samples odd-degree slot-polynomials over R_w in a valuation-controlled manner and keeps the odd part of the dynamics locally permutation-like.

2.3 HHS-like commutative actions

The HHS paradigm can be stated abstractly as follows. Let X be a public set and let $\mathcal{A}$ be a family of public self-maps of X indexed by hidden data. The goal is to identify a large commutative subfamily

$$\mathcal{C} \subseteq \mathcal{A}$$

with the property that, for any two secret elements $A, B \in \mathcal{C}$ and any public base point $g \in X$,

$$A(B(g)) = B(A(g)).$$

This is the only structural requirement needed for a Diffie–Hellman-like exchange; no ambient group law on X is required.[Cou06, CLM+18]

3 From scalar 2-adic CQA to a 2D operator action

3.1 Scalar family and macro-step ancestry

The scalar baseline starts from a public family map of arity m over R_w,

$$f(x_1, \ldots, x_m) = \prod_{i=1}^{m} P_i(x_i) \pmod{2^w},$$

where each slot polynomial P_i is an odd-degree 2-adic polynomial sampled in the conservative regime. Secret tuples are then turned into unary self-maps by a cyclic macro-step wiring: the current state is inserted into one slot, the local-law output is fed forward, and the secret tuple is rotated between micro-steps.

The 2D design inherits this exact ancestry. It does *not* replace the scalar macro-step machinery; instead, it uses the scalar mechanism twice to extract two scalar coefficients, which are then assembled into a commuting 2×2 operator.

3.2 Reduced scalar action with public tags

In the 2D proposal, the public family still has arity m, but the secret vector has length $m - 2$. One slot is used for a live scalar state x, and one extra slot is fixed to a public tag τ. Thus the reduced scalar macro-step is built from the argument list

$$[k_1, \ldots, k_{m-2}, x, \tau],$$

with the live state reinserted at moving positions and the tag held fixed in the final slot.

This gives a reduced branch extractor

$$T_{K,\tau}(x) : R_w \to R_w,$$

obtained by iterating the reduced macro-step `scalar_rounds` times.

4 The current standalone 2D CQA KEX design

Construction 4.1 (Standalone 2D CQA KEX)**.** *Fix a dyadic ring R_w, an arity $m \geq 4$, a public family of slot-polynomials, a public family seed, a public tag pair $(\tau_0, \tau_1) \in R_w^2$, two public coefficient-extraction seeds $(xseed_0, xseed_1) \in R_w^2$, and a public base vector*

$$G = (g_1, g_2) \in R_w^2.$$

Let K be a secret tuple of length $m - 2$.

Define the two scalar coefficients by

$$\lambda_0(K) = T_{K,\tau_0}(xseed_0), \qquad \lambda_1(K) = T_{K,\tau_1}(xseed_1).$$

The associated 2×2 operator is

$$M_K = \lambda_0(K)I + \lambda_1(K)S, \qquad S = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Equivalently,

$$M_K = \begin{bmatrix} \lambda_0(K) & \lambda_1(K) \\ \lambda_1(K) & \lambda_0(K) \end{bmatrix}.$$

For a public vector-round parameter $r \geq 1$, define the public key by

$$P_K := M_K^r(G).$$

Given two secret tuples A, B, the two parties publish

$$A_{\text{pub}} = M_A^r(G), \qquad B_{\text{pub}} = M_B^r(G),$$

and derive the shared secret by

$$\text{ss}_A = M_A^r(B_{\text{pub}}), \qquad \text{ss}_B = M_B^r(A_{\text{pub}}).$$

Whenever the two operators commute, the shared secret agrees:

$$\text{ss}_A = \text{ss}_B = M_A^r M_B^r(G).$$

Remark 4.2 (Anchored versus perturbed coefficient extraction)**.** *Setting*

$$xseed_0 = xseed_1 = 1$$

recovers the previous anchored version of the design. Randomizing $(xseed_0, xseed_1)$ over R_w^2 gives the Campaign 3 perturbation sweep used in the latest experiments.

4.1 Transcript-bound session key derivation

The raw shared secret is a 2D vector in R_w^2. The standalone script serializes the two coordinates, concatenates the public transcript

$$A_{\text{pub}} \parallel B_{\text{pub}},$$

and derives a session key using HKDF-style HMAC-SHA3-256. This does not solve authentication, but it ensures that the final session key is transcript-bound and does not directly expose the raw 2-adic vector.

5 Diagnostics and collapse notions

5.1 Direct one-public-vector recovery test

For the swap-family operator, one public vector gives the linear system

$$A_{\text{pub}} = M_A G, \qquad M_A = \begin{bmatrix} \lambda_0 & \lambda_1 \\ \lambda_1 & \lambda_0 \end{bmatrix}.$$

This is directly invertible whenever

$$\Delta(G) = g_1^2 - g_2^2$$

is invertible modulo 2^w. Thus the obvious recovery test is controlled by the parity of $\Delta(G)$.

Remark 5.1 (Odd-G policy)**.** *If both g_1 and g_2 are odd, then $g_1^2 \equiv g_2^2 \equiv 1 \pmod 8$, so*

$$\Delta(G) = g_1^2 - g_2^2$$

is automatically even (indeed divisible by 8), and therefore noninvertible modulo 2^w. This blocks the direct one-public-vector recovery route built into the current diagnostic.

5.2 Swap-basis collapse diagnostics

In the symmetric/antisymmetric basis,

$$s = x + y, \qquad d = x - y,$$

the operator M_K has eigenvalues

$$\mu_+(K) = \lambda_0(K) + \lambda_1(K), \qquad \mu_-(K) = \lambda_0(K) - \lambda_1(K).$$

These branch values determine whether the 2D action stays genuinely two-dimensional or collapses to a line.

The current diagnostics therefore monitor:

- per-key scalar collapse of M_K^r,
- one-branch and two-branch zero behavior in the swap basis,
- pairwise/shared-secret collapse classes:

`full_2d`, `symmetric_1d`, `antisymmetric_1d`, `zero_collapse`.

Definition 5.2 (Pairwise/shared-secret collapse)**.** *For two users A, B and public base vector $G = (g_1, g_2)$, define*

$$s_0 = g_1 + g_2, \qquad d_0 = g_1 - g_2.$$

After r rounds, the shared-secret channels are

$$s_{\text{fin}} = \mu_{+,A}^r \mu_{+,B}^r s_0, \qquad d_{\text{fin}} = \mu_{-,A}^r \mu_{-,B}^r d_0.$$

The shared secret is classified as:

- ***full_2d*** *if $s_{\text{fin}} \neq 0$ and $d_{\text{fin}} \neq 0$,*
- ***symmetric_1d*** *if $s_{\text{fin}} \neq 0$ and $d_{\text{fin}} = 0$,*
- ***antisymmetric_1d*** *if $s_{\text{fin}} = 0$ and $d_{\text{fin}} \neq 0$,*
- ***zero_collapse*** *if $s_{\text{fin}} = 0$ and $d_{\text{fin}} = 0$.*

5.3 Scalability of the 2D design

The present standalone design is written in a visibly two-dimensional form, but the construction principle is not inherently restricted to 2×2 operators. The current choice

$$G = (g_1, g_2) \in R_w^2$$

should be read as the smallest nontrivial public orbit vector for which the hidden coefficient family and the public orbit can both escape exact one-dimensional collapse under the present diagnostics.

More generally, one may choose a public orbit vector of length

$$\mathrm{Len}(G) = n_G, \qquad n_G \in \{3, 4, \dots\},$$

and then replace the current reduced-extractor lift by a family of higher-dimensional commuting operators acting on

$$R_w^{n_G}.$$

A natural design path is to pass from the present m-ary local law to an $(m + n_G)$-ary local law, using the same cyclic-wiring philosophy and the same 1D CQA tricks for state reinsertion, tag placement, and valuation control. In this way, the current two-branch coefficient extraction may be generalized to an n_G-branch coefficient family, producing larger operator matrices and potentially richer commutative substructures.

Conceptually, the present 2D design should therefore be viewed as the first nontrivial point in a broader hierarchy:

$$R_w \rightsquigarrow R_w^2 \rightsquigarrow R_w^{n_G},$$

where the ambient local law grows from arity m to arity $(m + n_G)$ and the visible action grows from scalar maps to matrix actions of increasing dimension. The main open question is whether the empirical commutative window observed for the current swap-family design extends to these higher-dimensional lifts, or whether new collapse modes appear as n_G grows.

6 Latest empirical picture

The latest experiments were performed on the current 2D swap-family standalone scaffold and its batch harnesses. The important qualitative findings are as follows.

6.1 Anchored odd-G baseline at tiny width

At tiny width $w = 16$, the anchored odd-G baseline already showed substantial collapse. In the anchored regime $(xseed_0, xseed_1) = (1, 1)$ with odd public base-vector coordinates and random public tags, the observed shared-secret behavior was approximately split between full 2D outputs and collapse events, with strong concentration on the symmetric line and the zero vector. In particular, the anchored odd-G baseline exhibited a substantial `symmetric_1d` rate and a nontrivial `zero_collapse` rate, while the obvious direct recovery test remained blocked because $\Delta(G)$ was noninvertible.

6.2 Campaign 3 perturbation sweep at $w = 16$

The Campaign 3 perturbation sweep randomized $(xseed_0, xseed_1)$ over R_w^2 while keeping the odd-G policy. This showed clearly that the old anchor $x = 1$ is not neutral at toy width.

Qualitatively, the perturbation sweep at $w = 16$ produced:

- a noticeably *higher* `full_2d` rate than the anchored baseline,
- a dramatic *reduction* in the `symmetric_1d` collapse rate,
- the appearance of a small but nonzero `antisymmetric_1d` rate,
- a modest increase in `zero_collapse`,
- and a higher per-key scalar-collapse rate.

Thus the $x = 1$ anchor materially changes the *distribution of collapse modes* at toy width. The toy-width collapse itself remains real, but the anchor strongly favors the symmetric-line failure mode.

6.3 Clean odd-G regime at moderate and large widths

The high-width picture is much better. In the odd-G regime, long runs for widths

$$112, 128, 144, 160, 196, 212, 224, 228, 240, 256$$

showed perfect commutativity and no observed pairwise/shared-secret collapse in the uploaded long-run summaries. In these runs, no direct one-public-vector recovery event was observed either, consistent with the odd-G policy.

Moreover, the Campaign 3 perturbation sweep did *not* destroy this clean regime. Randomized-`xseed` runs at width 112 remained fully clean in the tested sample, and width 128 remained fully clean even at 10^5 trials.

Remark 6.1 (Current empirical threshold)**.** *The latest evidence therefore supports a practical distinction:*

- *the severe collapse phenomena are presently a toy-width issue, clearly visible at $w = 16$;*
- *in the currently tested odd-G regime, no collapse has been observed in the high-width window beginning already at $w = 112$.*

This is empirical rather than proved, but it is the most important current experimental fact about the present 2D design.

6.4 Conjectured one-wayness of $\mathcal{A}_K(V) := M_K^r(V)$ (revised)

A central unresolved question is whether the public orbit map

$$\mathcal{A}_K(V) := M_K^r(V)$$

is one-way in any meaningful cryptographic sense when the adversary is given only the visible public orbit point

$$P_K = \mathcal{A}_K(G) = M_K^r(G)$$

and the public system parameters. In the current standalone design, the published object is the vector $P_K \in R_w^2$, not the matrix M_K itself. Thus any proposed attack must begin from the actual leakage

$$P_K \in R_w^2,$$

rather than from a hypothetical public exposure of all entries of M_K or M_K^r.

There are at least three distinct inversion goals that should be separated carefully.

(i) recover the exact secret tuple K;
(ii) recover the visible extracted coefficient pair $(\lambda_0(K), \lambda_1(K))$;
(iii) recover any hidden representation K' that induces the same public action on relevant orbit points.

These goals are not equivalent. Even in toy models, the public action may identify hidden parameters only up to symmetries; for example, in the swap-family toy inversion harness with even r, the coefficient pairs (λ_0, λ_1) and (λ_1, λ_0) give the same public orbit value, so exact ordered-pair recovery is impossible from the orbit point alone. Accordingly, any serious one-wayness discussion should be phrased in terms of recovering an *action-equivalent hidden representation*, not merely the exact original tuple.

From the implementation point of view, the attacker faces the composed inverse problem

$$K \longmapsto (\lambda_0(K), \lambda_1(K)) \longmapsto M_K \longmapsto M_K^r(G).$$

The first map is the reduced-extractor map

$$K \longmapsto \big(T_{K,\tau_0}(xseed_0),\ T_{K,\tau_1}(xseed_1)\big),$$

which is itself highly structured and 2-adic. The last map is the public orbit evaluation at G. The current odd-G policy is specifically designed so that the direct linear inversion route based on

$$\Delta(G) = g_1^2 - g_2^2$$

is blocked, because $\Delta(G)$ is noninvertible modulo 2^w when both g_1 and g_2 are odd. Therefore the simplest one-public-vector recovery attack is intentionally unavailable in the current mainline regime.

It is still reasonable to catalogue several possible attack families, but they should be stated only as *conditional hypotheses* rather than as established attacks.

R1. Brute-force or meet-in-the-middle on the effective image. If the image of the reduced-extractor map were shown to lie in a significantly smaller subset of R_w^2, then one could enumerate candidate coefficient pairs or candidate secret tuples and check whether

$$M_{K'}^r(G) = P_K.$$

This is a coherent attack model for toy widths and low-entropy images, but it is not currently a demonstrated attack on the empirically clean high-width regime.

R2. 2-adic polynomial solving from the visible orbit equations. Since only the two coordinates of P_K are public, any algebraic recovery method must work from two orbit equations rather than from a fully visible public matrix. A plausible line of attack is to rewrite these coordinates in the symmetric/antisymmetric basis and attempt Hensel-style lifting or other 2-adic solving methods for the hidden branch quantities. At present, however, no explicit general solver is known for the current standalone design.

R3. Lattice-style recovery heuristics. It is natural to ask whether the 2-adic structure can be converted into a small-root or short-vector problem for the hidden branch data. Such an idea could have merit, but no explicit lattice embedding, determinant estimate, or success condition is presently known for the real public-key interface $P_K \in R_w^2$. Therefore "LLL attack" should currently be understood as a research direction rather than as an established break.

R4. Precomputation attacks on low-entropy images. Rainbow-table or collision-style attacks are meaningful only if the public-action image is substantially smaller than the ambient ring. The current experiments do not establish such low-entropy image collapse in the practical-width odd-G regime; on the contrary, the strongest current evidence is empirical non-collapse.

For these reasons, the present project does not yet claim a theorem of one-wayness. The current working hypothesis is more modest: the high-width odd-G regime may support a nontrivial preimage-resistance problem because the attacker must simultaneously cope with hidden reduced-extractor preimages, iterated swap-family operator structure, and the absence of the direct determinant inversion route. Turning this heuristic into a precise hardness statement remains open.

6.5 Recovering the shared secret from two public vectors (revised)

In the actual standalone protocol, an eavesdropper sees the two public orbit vectors

$$A_{\text{pub}} = M_A^r(G), \qquad B_{\text{pub}} = M_B^r(G),$$

not the matrices M_A, M_B, or their powers. Therefore the shared secret

$$\text{ss} = M_A^r(B_{\text{pub}}) = M_B^r(A_{\text{pub}})$$

is not directly computable from the transcript unless the adversary first reconstructs enough hidden information to emulate one of the two actions. This point is conceptually important: the intended functionality of the protocol is that each honest party can evaluate its own hidden action on the other party's public orbit vector, but an external observer cannot do so from public information alone.

A realistic passive attack on the shared secret must therefore reduce to one of the following tasks.

S1. Recover an action-equivalent representation for one party. If an adversary can recover any hidden representation K'_A or K'_B such that

$$\mathcal{A}_{K'_A}(V) = \mathcal{A}_{K_A}(V) \qquad \text{or} \qquad \mathcal{A}_{K'_B}(V) = \mathcal{A}_{K_B}(V)$$

on relevant public inputs, then the shared secret follows immediately by applying the recovered action to the opposite public key. Thus shared-secret recovery reduces to action recovery for at least one participant.

S2. Jointly recover the hidden coefficient data for both parties. One may try to use the pair of equations

$$A_{\text{pub}} = M_A^r(G), \qquad B_{\text{pub}} = M_B^r(G)$$

to recover the branch data of both parties simultaneously. This is mathematically plausible in toy or low-entropy models, but no explicit efficient method is presently known for the current odd-G high-width regime.

S3. Exploit structural distinguishers or low-entropy image collapse. If future work were to show that public keys occupy a significantly smaller or highly structured subset of R_w^2, then transcript-level precomputation or distinguishing attacks might help reduce shared-secret recovery to a lookup problem. At present, the strongest current empirical evidence points in the opposite direction for the practical-width odd-G regime: no such collapse has been observed in long runs.

S4. Use a yet-to-be-formulated 2-adic lattice or lifting attack. It remains possible that the pair of public orbit equations hides a useful 2-adic short-vector or small-root structure. However, as in the one-wayness discussion above, no explicit lattice construction or success theorem is currently known for the real public-key interface.

The upshot is that recovering a shared secret from two public keys is presently not known to be easier than recovering an action-equivalent hidden representation for one of the parties. Consequently, the natural defensive guidance remains unchanged: use transcript binding, prefer ephemeral keys, and treat shared-secret recovery from two public vectors as a primary cryptanalytic target for future work rather than as a solved attack.

7 Open problems

Q1. Transition-width problem. Where is the true transition between the toy-width collapse regime and the stable odd-G regime? The present evidence strongly separates $w = 16$ from the currently clean window beginning at $w = 112$, but the intermediate range remains to be mapped carefully.

Q2. Family-seed robustness. The strongest current long runs are still concentrated around one family seed. How universal is the clean odd-G behavior across many public families and nearby implementation variants?

Q3. Why does the $x = 1$ anchor bias toy-width collapse? The perturbation sweep shows that the anchor strongly favors symmetric-line collapse at $w = 16$. Is there a closed-form 2-adic explanation for this asymmetry in the symmetric/antisymmetric basis?

Q4. Can the clean regime be proved? Can one formulate a rigorous criterion ensuring that the pairwise/shared-secret action remains outside exact one-dimensional collapse for the current swap-family operator at sufficiently large width?

Q5. Security beyond the obvious recovery test. Blocking the direct $\Delta(G)$ route is only the first line of defense. What are the right formal security notions for recovering an action-equivalent hidden representation from a public orbit vector?

Q6. Recovering a shared secret from two public keys. Is there any method substantially easier than full action recovery for computing

$$\mathrm{ss} = M_A^r(B_{\mathrm{pub}}) = M_B^r(A_{\mathrm{pub}})$$

from the two visible public orbit vectors and the public parameters alone?

Q7. Distinguishing public keys from random points in R_w^2. Do public keys occupy a statistically distinguishable subset of R_w^2 in the practical-width odd-G regime, or does the current non-collapse evidence suggest pseudorandom-looking public orbit points?

Q8. CPA/CCA conversion and authenticated encapsulation. Even if the raw commutative action remains viable as a shared-secret primitive, what are the right CPA and CCA transformations for turning it into a standard PKE or KEM object with meaningful chosen-plaintext and chosen-ciphertext security?

Q9. Can a genuine 2-adic lattice formulation be written down? Several heuristic attack ideas suggest possible LLL- or lifting-based approaches, but no explicit lattice basis, determinant bound, or short-vector theorem is currently known for the real public-key interface. Is there in fact a useful 2-adic lattice problem hidden in the current swap-family orbit equations?

8 Conclusions (for the state of the project up to 12 March 2026)

The current state of the project is now more coherent than before. The 2D design is no longer merely a broad search over schedules: it is a concrete standalone KEX scaffold built from reduced scalar extractors, public tags, public xseeds, a swap-family operator, an odd-coordinate public base vector, and a transcript-bound KDF.

The latest experiments sharpen the empirical picture considerably. The rounds≥ 2 semantics are fixed; the odd-G policy blocks the current direct one-public-vector recovery condition; the old $x = 1$ anchor materially biases collapse at toy width; yet the clean high-width odd-G regime appears stable both in the anchored and randomized-`xseed` variants. The main challenge is now theoretical and cryptanalytic at the same time: to explain why the high-width regime behaves so much better than the toy-width regime, to formulate the correct one-wayness and shared-secret-recovery problems for the real public-key interface, and to determine whether any genuine 2-adic algebraic or lattice-based attack exists against the current design.

References

[CLM+18] Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. Csidh: An efficient post-quantum commutative group action. In *Advances in Cryptology – ASIACRYPT 2018*, volume 11274 of *Lecture Notes in Computer Science*, pages 395–427. Springer, 2018. Also available as IACR ePrint 2018/383.

[Cou06] Jean-Marc Couveignes. Hard homogeneous spaces. Technical Report 2006/291, IACR Cryptology ePrint Archive, 2006.

[DF17] Luca De Feo. Mathematics of isogeny based cryptography. *CoRR*, abs/1711.04062, 2017.

[Gli26] Danilo Gligoroski. Commutative-quasigroup-actions (cqa) proof-of-concept. `https://github.com/danilo-gligoroski/Commutative-Quasigroup-Actions`, 2026. GitHub repository, accessed 2026-03-12.

[MŠG10] S. Markovski, Z. Šunić, and D. Gligoroski. Polynomial functions on the units of $\mathbb{Z}_{2^n}$. *Quasigroups and Related Systems*, 23(1):59–82, 2010.